

CSE-4301
Object Oriented Programming
2023-2024

Week-10

Stream and Files

-Faisal Hussain

Assistant Professor, Department Of Computer Science And Engineering, IUT

Stream

- A *stream* is a general name given to a flow of data
- cin and cout are stream objects
- Different stream class represent different kinds of data flow

Advantage over C style file function:

- Simplicity. Use of format specifier is not necessary
- Overload insertion(<<) and extraction (>>) operators,
- they are the best way to write data to files, and also to format data in memory



standard stream

- **standard streams** are **pre-connected input and output channels** between a **computer program and its environment** (typically a text terminal) when it begins execution.
- In C : The three I/O connections are called
 1. standard input (**stdin**)
 2. standard output (**stdout**)
 3. standard error (**stderr**).



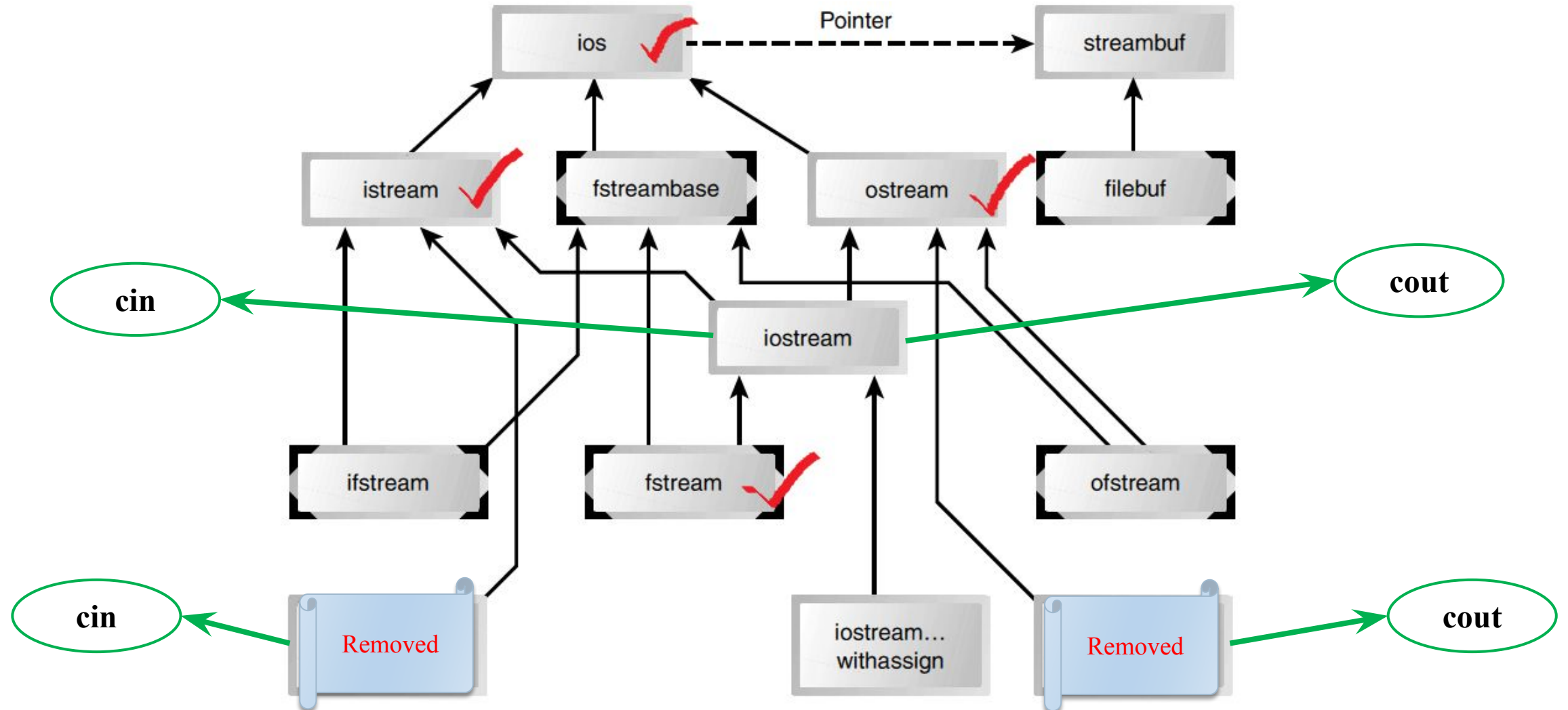
standard stream

□ In C++ : The four I/O connections are called

1. standard input (**cin**) **Default device : Keyboard**
2. standard output (**cout**) **Default device : Screen**
3. standard error (**cerr**) **Default device : Screen**
4. Buffered version of cerr (**clog**). **Default device : Screen**



Stream Class Hierarchy



ios Class

□ contains :

1. **Formatting flags**
2. **Error-status flags**
3. **File operation mode**

Formatting flags

- A set of `enum` definitions in `ios` class.
- They act as on/off switches
- That specify choices for various aspects of input and output format and operation.

Formatting flags

field	member constant	effect when set
<i>independent flags</i>	boolalpha	read/write bool elements as alphabetic strings (true and false).
	showbase	write integral values preceded by their corresponding numeric base prefix.
	showpoint	write floating-point values including always the decimal point.
	showpos	write non-negative numerical values preceded by a plus sign (+).
	skipws	skip leading whitespaces on certain input operations.
	unitbuf	flush output after each inserting operation.
	uppercase	write uppercase letters replacing lowercase letters in certain insertion operations.
<i>numerical base (basefield)</i>	dec	read/write integral values using decimal base format.
	hex	read/write integral values using hexadecimal base format.
	oct	read/write integral values using octal base format.
<i>float format (floatfield)</i>	fixed	write floating point values in fixed-point notation.
	scientific	write floating-point values in scientific notation.
<i>adjustment (adjustfield)</i>	internal	the output is padded to the <i>field width</i> by inserting <i>fill characters</i> at a specified internal point.
	left	the output is padded to the <i>field width</i> appending <i>fill characters</i> at the end.
	right	the output is padded to the <i>field width</i> by inserting <i>fill characters</i> at the beginning.

Formatting flags

- ❑ `cout.setf (ios::left);` *// left justify output text*
- ❑ `cout << “This text is left-justified”;`
- ❑ `cout.unsetf (ios::left);` *// return to default (right justified)*
- ❑ Many formatting flags can be set using manipulators

Manipulators

- ❑ Manipulators are formatting instructions inserted directly into a stream.
- ❑ **endl**: which sends a newline to the stream and flushes it:
- ❑ `cout << "To each his own." << endl << " ok";`
- ❑ `cout << setiosflags(ios::fixed) // use fixed decimal point
 << setiosflags(ios::showpoint) // always show decimal point
 << var;`

Manipulators

□ Two Variant

- No argument ios Manipulator
- ios Manipulator with argument

<i>Manipulator</i>	<i>Purpose</i>
ws	Turn on whitespace skipping on input
dec	Convert to decimal
oct	Convert to octal
hex	Convert to hexadecimal
endl	Insert newline and flush the output stream
ends	Insert null character to terminate an output string
flush	Flush the output stream
lock	Lock file handle
unlock	Unlock file handle



ios Manipulator with argument

<i>Manipulator</i>	<i>Argument</i>	<i>Purpose</i>
<code>setw()</code>	field width (int)	Set field width for output
<code>setfill()</code>	fill character (int)	Set fill character for output (default is a space)
<code>setprecision()</code>	precision (int)	Set precision (number of digits displayed)
<code>setiosflags()</code>	formatting flags (long)	Set specified flags
<code>resetiosflags()</code>	formatting flags (long)	Clear specified flags



ios Function

<i>Function</i>	<i>Purpose</i>
<code>ch = fill();</code>	Return the fill character (fills unused part of field; default is space)
<code>fill(ch);</code>	Set the fill character
<code>p = precision();</code>	Get the precision (number of digits displayed for floating-point)
<code>precision(p);</code>	Set the precision
<code>w = width();</code>	Get the current field width (in characters)
<code>width(w);</code>	Set the current field width
<code>setf(flags);</code>	Set specified formatting flags (for example, <code>ios::left</code>)
<code>unsetf(flags);</code>	Unset specified formatting flags
<code>setf(flags, field);</code>	First clear field, then set flags

istream function

<i>Function</i>	<i>Purpose</i>
>>	Formatted extraction for all basic (and overloaded) types.
<code>get(ch);</code>	Extract one character into <code>ch</code> .
<code>get(str)</code>	Extract characters into array <code>str</code> , until <code>'\n'</code> .
<code>get(str, MAX)</code>	Extract up to <code>MAX</code> characters into array.
<code>get(str, DELIM)</code>	Extract characters into array <code>str</code> until specified delimiter (typically <code>'\n'</code>). Leave delimiting char in stream.
<code>get(str, MAX, DELIM)</code>	Extract characters into array <code>str</code> until <code>MAX</code> characters or the <code>DELIM</code> character. Leave delimiting char in stream.
<code>getline(str, MAX, DELIM)</code>	Extract characters into array <code>str</code> , until <code>MAX</code> characters or the <code>DELIM</code> character. Extract delimiting character.
<code>putback(ch)</code>	Insert last character read back into input stream.
<code>ignore(MAX, DELIM)</code>	Extract and discard up to <code>MAX</code> characters until (and including) the specified delimiter (typically <code>'\n'</code>).
<code>peek(ch)</code>	Read one character, leave it in stream.
<code>count = gcount()</code>	Return number of characters read by a (immediately preceding) call to <code>get()</code> , <code>getline()</code> , or <code>read()</code> .
<code>read(str, MAX)</code>	For files—extract up to <code>MAX</code> characters into <code>str</code> , until EOF.
<code>seekg()</code>	Set distance (in bytes) of file pointer from start of file.
<code>seekg(pos, seek_dir)</code>	Set distance (in bytes) of file pointer from specified place in file. <code>seek_dir</code> can be <code>ios::beg</code> , <code>ios::cur</code> , <code>ios::end</code> .
<code>pos = tellg(pos)</code>	Return position (in bytes) of file pointer from start of file.



ostream function

<i>Function</i>	<i>Purpose</i>
<<	Formatted insertion for all basic (and overloaded) types.
put(ch)	Insert character ch into stream.
flush()	Flush buffer contents and insert newline.
write(str, SIZE)	Insert SIZE characters from array str into file.
seekp(position)	Set distance in bytes of file pointer from start of file.
seekp(position, seek_dir)	Set distance in bytes of file pointer, from specified place in file. seek_dir can be ios::beg, ios::cur, or ios::end.
pos = tellp()	Return position of file pointer, in bytes.



Error Status Bits

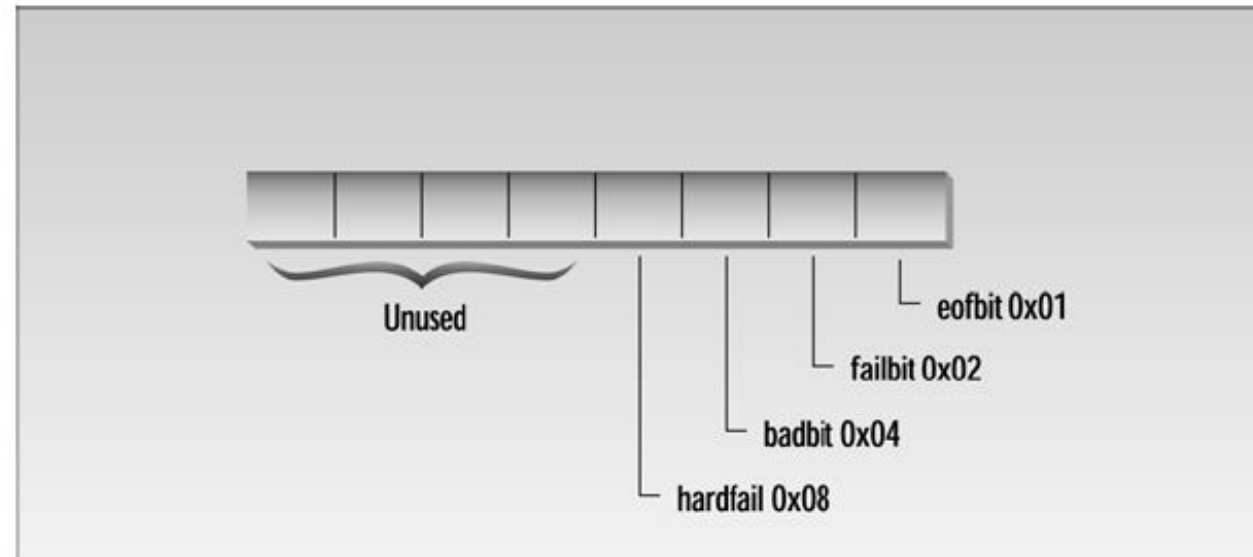
- The stream error-status flags constitute an `ios` enum member that reports errors that occurred in an input or output operation

<i>Name</i>	<i>Meaning</i>
<code>goodbit</code>	No errors (no flags set, value = 0)
<code>eofbit</code>	Reached end of file
<code>failbit</code>	Operation failed (user error, premature EOF)
<code>badbit</code>	Invalid operation (no associated streambuf)
<code>hardfail</code>	Unrecoverable error



Error Status Bits

<i>Function</i>	<i>Purpose</i>
<code>int = eof();</code>	Returns true if EOF flag set
<code>int = fail();</code>	Returns true if failbit or badbit or hardfail flag set
<code>int = bad();</code>	Returns true if badbit or hardfail flag set
<code>int = good();</code>	Returns true if everything OK; no flags set
<code>clear(int=0);</code>	With no argument, clears all error bits; otherwise sets specified flags, as in <code>clear(ios::failbit)</code>



Disk File I/O with Streams

- ifstream – For taking Input
- fstream – For Input and Output both
- ofstream – For Output

- They are declared in <fstream> header file



C style file I/O

- `fread()`

- `fwrite()`

... will still works in `c++`

...be careful about mixing the old C functions with C++ streams. They don't always work together gracefully, although there are ways to make them cooperate.



Writing Data

- ❑ `ofstream out("filename");`
- ❑ various resources for the file, and accesses or *opens* the file of that name on the disk.
- ❑ If the file is not present, one will be created at the beginning.
- ❑ `out<<"Hello";`
- ❑ When the program terminates, the `out` object goes out of scope. This calls its destructor, which closes the file, No need to close the file explicitly.



Writing Data

- Since numbers are stored as a sequence of characters, rather than as a fixed-length field, you must separate numbers with nonnumeric characters.



Reading Data

- ❑ `ifstream in("filename");`
- ❑ Blanks in string will be assumed as different string.
- ❑ `char ch; int j; double d; string str1, str2;`
- ❑ `in >> ch >> j >> d >> str1 >> str2;`
- ❑ Use `getline()` to read one line at a time.



Reading from File

- ❑ Detecting end of file – `eof()` member function
- ❑ The EOF is a signal sent to the program from the operating system when there is no more data to read
- ❑ `while( infile.good() ) // until any error encountered`
- ❑ Using ios error flags
- ❑ `while( infile ) // until any error encountered`
- ❑ If everything is going well, the object returns a nonzero value



Character I/O

- `put()` and `get()` functions, which are members of `ostream` and `istream` can be used to output and input single character.



Binary I/O

- To store large amount of numerical data it is more efficient to use binary I/O.
- In this form data is stored as like RAM.
- In binary I/O int -> 4 bytes. But text version “12345” -> 5 bytes

- `ofstream os(“edata.dat”, ios::binary);`
- For Binary I/O we will use
- `write()`, a member of `ofstream`; and
- `read()`, a member of `ifstream`
 - parameters -> address of the data buffer and its length.
 - address must be cast, using `reinterpret_cast`, to type `char*`,



Object I/O

- ❑ Writing an object, generally use binary mode, `write()`.
- ❑ This writes the same bit configuration to disk that was stored in memory, and ensures that numerical data contained in objects is handled properly
- ❑ Reading an object back, use `read()` [file is in binary mode]



fstream class and open()

- ❑ `fstream` class is derived from `iostream` class
- ❑ `fstream` object can take both input and output.
- ❑ `open(file_name,mode)` function can be used to open a file. First parameter is a string and second parameter is the mode of opening.
- ❑ The vertical bars between the flags cause the bits representing these flags to be logically combined into a single integer, so that several flags can apply simultaneously.



Mode bit

<i>Mode Bit</i>	<i>Result</i>
in	Open for reading (default for ifstream)
out	Open for writing (default for ofstream)
ate	Start reading or writing at end of file (AT End)
app	Start writing at end of file (APPend)
trunc	Truncate file to zero length if it exists (TRUNCate)
nocreate	Error when opening if file does not already exist
noreplace	Error when opening for output if file already exists, unless ate or app is set
binary	Open file in binary (not text) mode

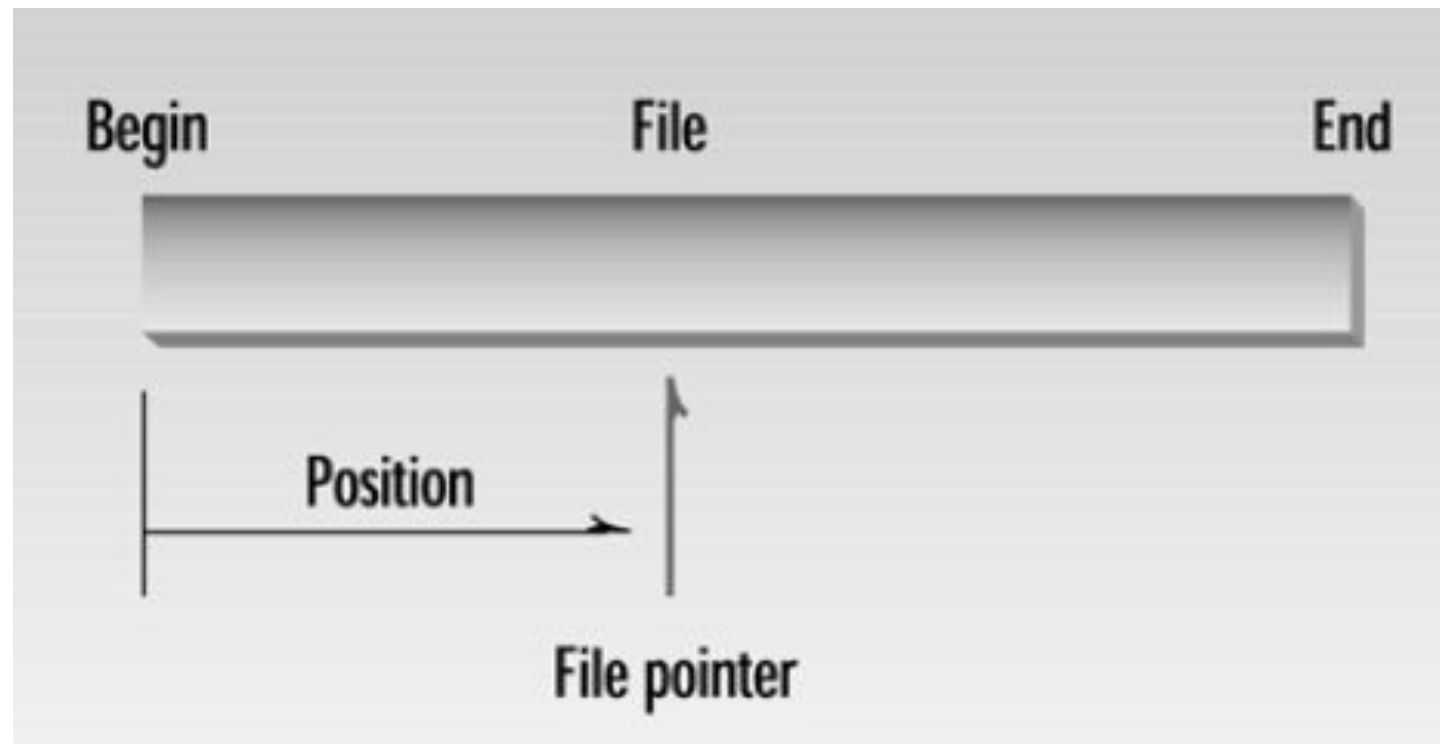


File Pointer

- File object has 2 integer values
 - *get pointer (current get position)*
 - *put pointer (current put position)*
- read from and write to arbitrary position
 - *Set and examine the get pointer*
 - *seekg()*
 - *tellg()*
 - *Set and examine the put pointer*
 - *seekp()*
 - *tellp()*



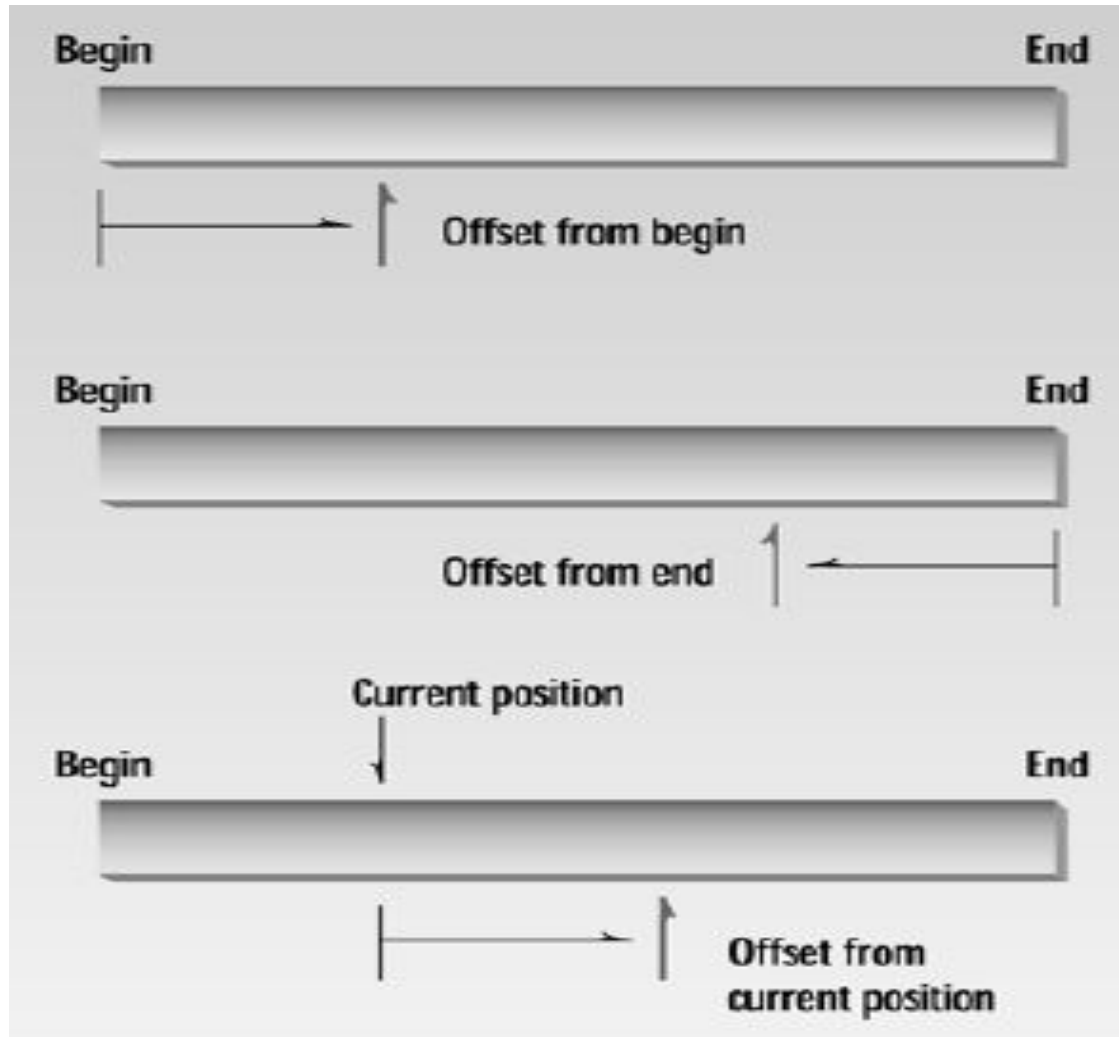
Specifying the position



- `seekg(absolution_position_in_byte)`
- `seekg(0)` -> the start of the file is byte 0



Specifying the Offset



- There are three possibilities for the second argument: beg is the beginning of the file (`ios::beg`), cur is the current pointer position (`ios::cur`), and end is the end of the file (`ios::end`).
- `seekg(-10, ios::end)`
- `tellg()` returns the current position of get pointer

Error Handling in File I/O

- ❑ A file that you think exists may not,
- ❑ or a filename that you assume you can use for a new file may already apply to an existing file.
- ❑ Or there may be no more room on the disk, or no disk in the drive, and so on.



Objects that Read and Write Themselves

- May be two member functions
 - `diskOut()` to write on file
 - `diskIn()` to read from file



Classes that Read and Write Themselves

- Using static function
 - Write all objects using the static member function
 - It can access an array of pointers to the objects which can be stored as static data.



Overloading the Extraction and Insertion Operators

- We can overload the extraction and insertion operators so they work with the display and keyboard (cout and cin) alone
- `friend istream& operator >> (istream& s, Distance& d)`
- `friend ostream& operator << (ostream& s, Distance& d)`



Acknowledgement

□ Ashraful Alam

Assistant Professor,
Department of Computer Science Engineering
Islamic University of Technology

